



如何基于 Pi 开发自己的 Agent 项目

01 看懂 Pi：选对开发层，跑通第一个 Agent

大家好，我是居丽叶。

在[上一篇](#)里，我们分层看了 Pi 的包结构、Context 和 Agent Loop，看过模型生成一条 Tool Call 之后，Pi 怎样执行工具，再把结果发回给模型。

本篇，就把这个运行过程接进自己要开发的 Agent 中。

我们的目标是建一个 `pi-minimal` 目录，装好依赖、连接模型、创建 Agent，然后看到第一条流式回复打在终端上。先把一个 Agent 跑通，后面几篇讲解中再逐步加入制度检索工具、输出校验，以及多个 Agent 之间的协作。

但打开编辑器之前，还要做一个选择：Pi 提供 CLI、SDK、RPC，底下又有 Agent Core。如果我们要开发自己的 Agent，应该从哪里接入？

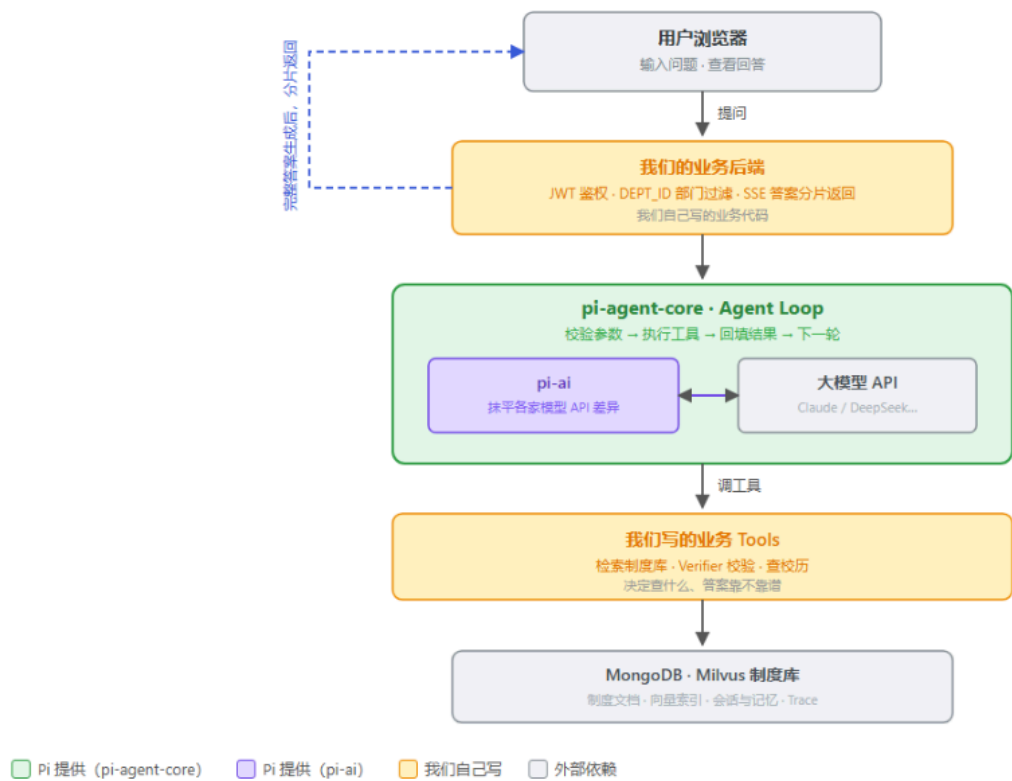
一、先选开发层，再选调用入口

[上一篇](#)按包的职责认识 Pi。到了开发阶段，我们把它们放回一个有业务场景的需求里：例如在校内行政 Agent 中，用户在网页上问“学校的差旅报销需要哪些材料”，后端要确认用户身份、查询制度、组织回答，再把生成过程传回网页。

这条链路里有几件不同的事：

- **接模型**：底下不管是 Anthropic 还是 DeepSeek，各家 API 协议都不一样，`pi-ai` 提供统一的模型调用接口，本例通过 `streamSimple` 发起流式请求。

- **跑循环**：模型说"我要调 `retrieve_documents`"，就按参数 schema 校验、执行工具、把结果塞回上下文、再发下一轮请求——这个来回打转的过程，是 `pi-agent-core` 的 Agent loop 在管。
- **定业务规则**：谁能查哪些制度（JWT 鉴权 + 部门权限）、检索时按 `DEPT_ID` 过滤、答案过 Verifier 这一关、trace 和反馈存 MongoDB——这些全是我们自己的代码，Pi 不掺和。
- **把过程给人看**：Pi 会发出文本增量、工具调用等事件，应用可以接收这些事件并展示运行过程。本文先把文本增量打印到终端，后续再讨论怎样接到网页。



这样看，Pi 在业务系统里的位置就有了：我们借用模型接入和运行循环，把它们放在自己的后端与界面之间。

不过，同样是做一个校内的行政助手，交付要求可能差很多。我们开发者在接到需求的时候，通常需要先弄明白“要帮谁完成什么工作”，接着才决定用哪些包。这里继续拿差旅报销举例，看看需求怎样影响选择：

第一种，只需要把已经查到的资料整理成答案。 比如用户选好部门和报销类型，后端按固定规则查询制度，再把相关条款交给模型，生成一份材料清单。检索步骤已经由业务代码确定，模型只负责组织文字，这时可以先用 `pi-ai` 接模型，无需额外装配 Agent Loop。业务代码负责查对资料、校验结果；涉及金额、日期等确定规则，也可以继续用程序计算。

第二种，要交付一个围绕文件工作的助手。 比如工作人员把制度文件放进指定目录，希望助手自行查阅、对照、整理出报销指南，并且可以继续追问、修改产物。这时需要模型与工具反复交互，也需要工作目录和连续会话，可以优先评估 `pi-coding-agent`：复用它的文件工具、会话管理和资源加载能力，再按任务补充工具。虽然这个包的名称里有 coding，这些能力也能用于文件整理类业务；PDF 等格式是否需要额外解析，也要另行确认。

第三种，要把助手接进已有的业务产品。 比如全校师生从网页查询制度，不同部门有不同权限，回答要给出处，调用过程要留记录，后续还要查询报销进度。这时，工具操作的对象已经变成受权限保护的制度库和业务接口。对于我做的行政 Agent 这种已有后端、权限和数据库的项目，可以直接从 `pi-agent-core + pi-ai` 开始：只把允许执行的业务操作注册成工具，身份、权限、记录和结果校验继续由产品系统负责。

可以先按交付目标做一轮选择：

业务要交付什么	建议从哪里开始	选择依据
根据已提供的资料，生成摘要或材料清单	pi-ai	步骤固定，只需要模型生成
围绕指定目录查资料、整理文件，并支持连续追问和修改产物	pi-coding-agent	复用已有会话、文件工具与资源加载能力
在已有产品中加入带权限、业务接口和审计的 Agent	pi-agent-core + pi-ai	自己定义工具与执行边界，接入现有业务管理

 **先确定交付目标和业务边界，再决定复用多少 Pi 的现成能力。**

选好这一层之后，才轮到调用入口：

内部人员直接操作可以用 `pi` 交互模式；

脚本执行一次任务可以用 `pi -p`；

Node 应用嵌入现成会话可以用 SDK；

Python 等程序持续控制独立 Pi 进程，可以用 `pi --mode rpc`。

RPC 通过标准输入输出传送逐行 JSON 命令、响应和事件，需要应用管理子进程并处理事件。

本系列使用的是第三种：接入制度库、部门权限和业务校验的行政 Agent，从 `pi-agent-core + pi-ai` 开始。在我的行政 Agent 中，会话事件和摘要已经由业务后端保存，后续请求会读取它们重建上下文；异步作业队列和周期 Worker 也由业务代码管理。Pi 接入这套系统，负责模型与工具之间的执行循环，已有的会话、记忆和任务管理继续留在后端。

本文采用 `1.0.0`。这一版 Core 已移出此前实验性的 Harness、会话存储和持久化运行时等能力，下面使用的是保留下来的 `Agent` 与 Agent Loop 接口。

二、建一个 pi-minimal 工程

下面固定使用 `1.0.0`，运行环境为 Node.js `22.19.0` 或更高版本。

配套工程在 [code/pi-minimal](#) 目录里，包含依赖声明、完整脚本和运行说明。下载系列仓库后，可以直接进入该目录安装运行；想从空目录开始，也可以按下面的步骤自己创建代码文件。

先检查环境：

```
1 node --version
2 npm --version
```

在操作系统终端创建目录，并进入它：

代码块

```
1 mkdir pi-minimal
2 cd pi-minimal
```

本次示例使用 **TypeScript + Node.js**，后续再把它封装成服务，供 Python 后端通过 HTTP 调用。这里先省去 HTTP 服务、业务工具和多 Agent 编排，只保留模型装配、一个 Agent 和流式回复。

新建 `package.json`，内容如下。

代码块

```
1 {
2   "name": "pi-minimal",
3   "private": true,
4   "type": "module",
5   "engines": { "node": ">=22.19.0" },
6   "scripts": { "start": "node index.ts" },
7   "dependencies": {
8     "@earendil-works/pi-agent-core": "1.0.0",
9     "@earendil-works/pi-ai": "1.0.0"
10  }
11 }
```

安装依赖：

代码块

```
1 npm install --ignore-scripts
```

安装后会出现 `node_modules` 和 `package-lock.json`。

接着准备模型凭据。通过 `pi-ai` 注册 OpenAI 兼容 provider，再把 `streamSimple` 接进 Agent。我这里面默认模型设置的是 `deepseek-v4-flash`，你需要自己的可用 API Key，在运行脚本的同一个终端设置：

Windows PowerShell：

代码块

```
1 $env:DEEPSEEK_API_KEY = "替换为你的API Key"
```

macOS / Linux Bash 或 zsh:

代码块

```
1 export DEEPSEEK_API_KEY="替换为你的API Key"
```

这个示例由 provider 读取环境变量，不会自动复用 Pi CLI 的登录状态。

三、创建 Agent，接住第一条流式回复

先在 `pi-minimal` 目录下新建 `providers.ts`，负责配置模型接入：使用哪个模型、请求发到哪个地址、从哪里读取 API Key，以及按什么接口协议发送请求。

代码块

```
1 import { createModels, createProvider, envApiKeyAuth, type Model } from
  "@earendil-works/pi-ai";
2 import { openAICompletionsApi } from "@earendil-works/pi-ai/api/openai-
  completions.lazy";
3
4 export function buildModels() {
5   const baseUrl = process.env.DEEPSEEK_BASE_URL ?? "https://api.deepseek.com";
6   const modelId = process.env.DEEPSEEK_MODEL ?? "deepseek-v4-flash";
7   const model: Model<"openai-completions"> = {
8     id: modelId, name: "DeepSeek 对话",
9     api: "openai-completions", provider: "deepseek", baseUrl,
10    reasoning: false, input: ["text"],
11    cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
12    contextWindow: 128000, maxTokens: 32000,
13    compat: {
14      supportsDeveloperRole: false,
15      supportsReasoningEffort: false,
16      supportsStore: false,
17    },
18  };
19  const models = createModels();
20  models.setProvider(createProvider({
21    id: "deepseek", name: "DeepSeek", baseUrl,
22    auth: { apiKey: envApiKeyAuth("DeepSeek API key", ["DEEPSEEK_API_KEY"]) },
23    models: [model], api: openAICompletionsApi(),
24  }));
25  return { models, model: models.getModel("deepseek", modelId)! };
```

这里的窗口、输出上限和兼容选项沿用我自己的项目中的配置；`cost` 的零值用于占位，不代表调用免费。

再在同一目录下新建 `index.ts`，作为程序入口。它调用前面的 `buildModels()` 取得模型配置，创建 Agent，订阅回复事件，然后提交一个问题，把模型返回的文字逐段打印到终端：

代码块

```
1  import { Agent } from "@earendil-works/pi-agent-core";
2  import { buildModels } from "./providers.ts";
3
4  if (!process.env.DEEPSEEK_API_KEY) {
5    throw new Error("请先设置 DEEPSEEK_API_KEY");
6  }
7  const { models, model } = buildModels();
8
9  const agent = new Agent({
10    initialState: {
11      systemPrompt: "你是一个行政助手，用中文简短回答；没有制度资料时说明需要补充依据。",
12      model,
13      tools: [],
14    },
15    streamFn: models.streamSimple.bind(models),
16  });
17
18  const unsubscribe = agent.subscribe((event) => {
19    if (event.type === "message_update" &&
20      event.assistantMessageEvent.type === "text_delta") {
21      process.stdout.write(event.assistantMessageEvent.delta);
22    }
23  });
24
25  try {
26    await agent.prompt("查询学校的差旅报销制度，需要先提供哪些资料? ");
27    const last = agent.state.messages.filter(m => m.role === "assistant").at(-1);
28    if (!last || ["error", "aborted"].includes(last.stopReason)) {
29      throw new Error(last?.errorMessage ?? "请求未正常完成");
30    }
31    console.log();
32  } finally {
33    unsubscribe();
34  }
```

运行：

代码块

```
1 npm start
```

运行结果如下，它现在只能根据输入说明需要补充什么资料。我们还没有给它接制度库，因此还无法查到学校的正式规定：

```
PS D:\code\pi-minimal> npm start
> start
> node index.ts

我目前没有你校的差旅报销制度原文，需补充依据才能查询。请先提供：

1. 学校全称及财务/人事归口部门
2. 你的身份：教职工/学生/外聘等
3. 差旅类型：国内/国外、会议/培训/调研等
4. 想查的具体内容：交通、住宿、补贴、审批流程等
5. 适用时间或制度版本/文号
6. 如要套具体标准，还需出差时间、目的地、事由

如有校内OA、财务处链接或制度文号，也请一并提供。
```

上面的代码分成四步：`buildModels()` 准备模型连接，`new Agent()` 创建使用这个模型的 Agent，`subscribe()` 接收运行过程中的事件，`await agent.prompt()` 提交问题并等待这次运行结束。下面按执行顺序解释这四步。

1. buildModels：准备 DeepSeek 接入

`createModels()` 创建 provider 集合；`createProvider()` 把 DeepSeek 的模型描述、环境变量认证和 `openAICompletionsApi()` 组合起来；`setProvider()` 完成注册，`getModel()` 按 provider 与模型 ID 取得模型定义。

2. new Agent：把模型装进运行循环

`initialState` 提供初始指令、模型和工具列表；`streamFn` 把 provider 集合的流式调用接进 Loop。调用 `agent.prompt()` 后，Pi 会组织消息、请求模型、接收流式响应，并更新运行状态。

这里的 `tools: []` 特意留空。先把模型连接与事件订阅跑通，下一篇再加入业务工具，出错时就能分清问题发生在哪一步。

3. subscribe：把事件变成界面输出

序篇介绍过一次运行中的事件。本例没有工具调用，主要沿着下面这条路径结束：

代码块

```
1 agent_start → turn_start
```

```
2    → message_start / message_end (用户消息)
3    → message_start (模型消息)
4    → message_update × N
5    → message_end (完整模型消息)
6    → turn_end → agent_end
```

`message_update` 中也可能出现思考或工具调用相关增量，因此打印文本时继续筛选 `text_delta`。`process.stdout.write()` 把新增片段追加到终端；以后接网页，可以在同一位置把增量发给前端。

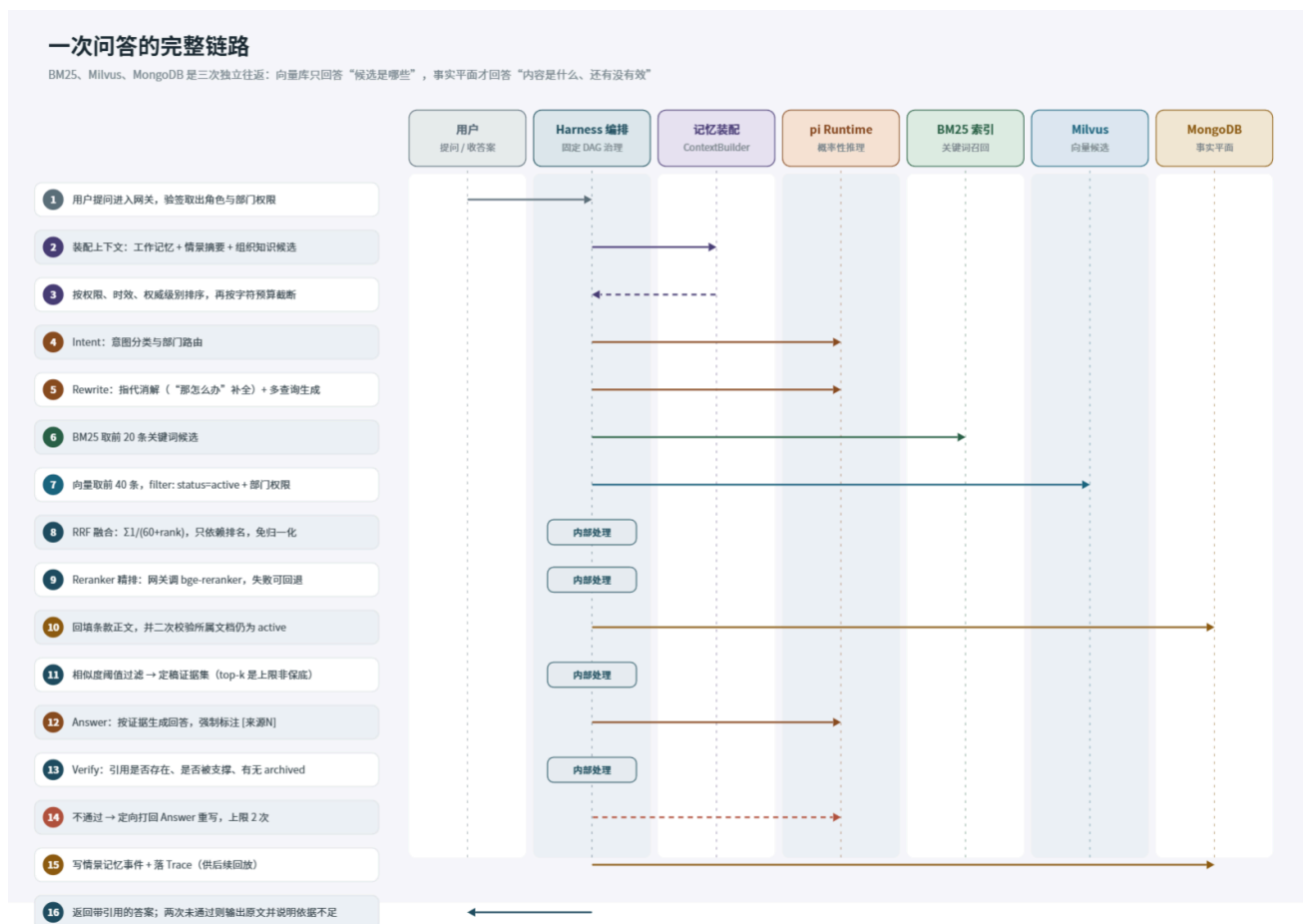
4. await prompt：等待运行结束，再检查结果

`await agent.prompt()` 等待这次运行完成。在当前无工具、无排队输入的示例里，通常是一轮模型回复；加入工具后，同一次 `prompt()` 可以包含多轮模型请求与工具执行。

四、从一个 Agent 接到自己的系统

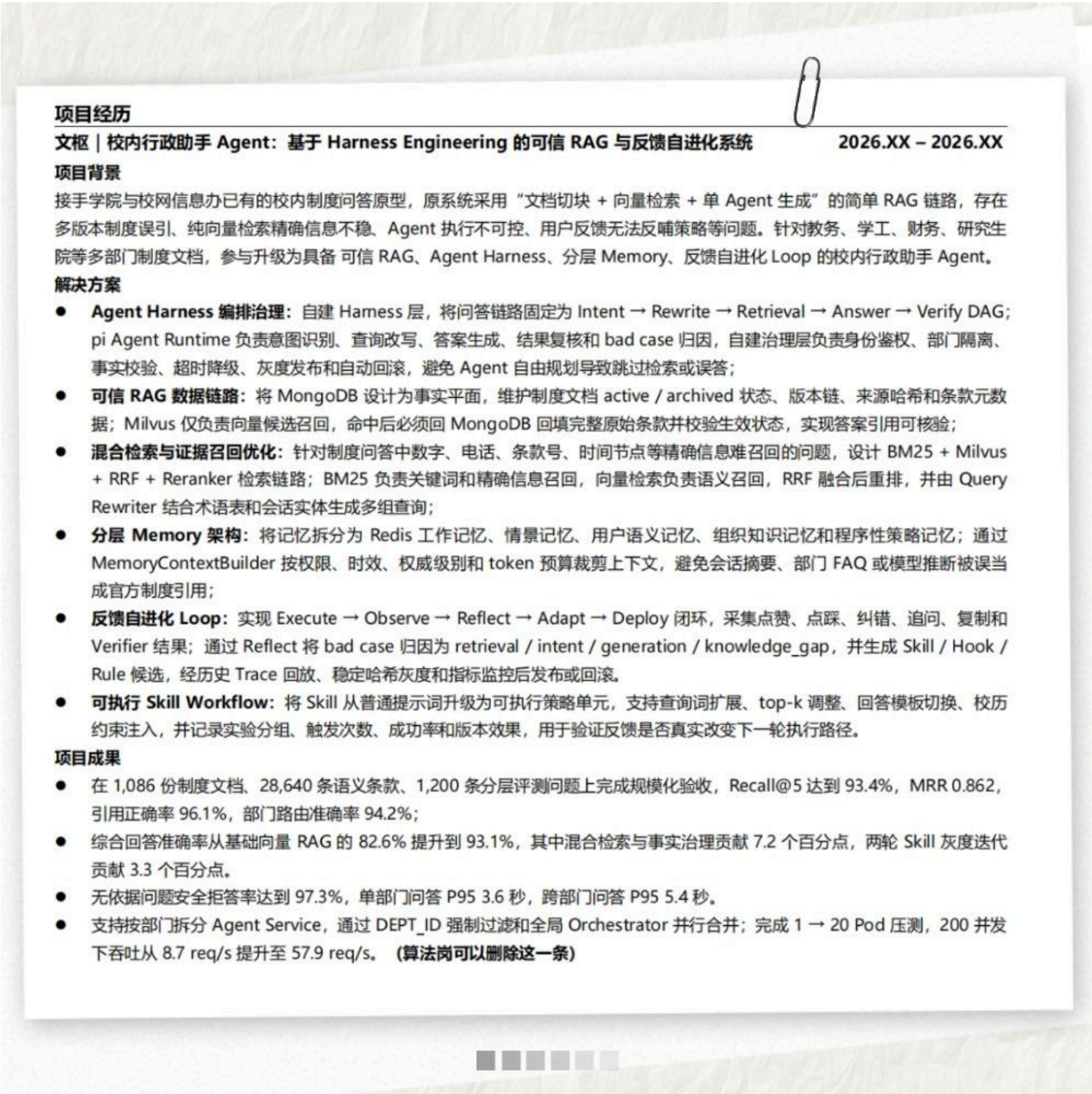
到这里，我们已经把 DeepSeek 接进 Pi，创建了一个 Agent，并在终端看到了它的回复。接下来，就可以沿着这套代码逐步加入工具和业务逻辑。

在我的行政 Agent 项目里，这个运行过程被放进了独立的 Node 服务：Python 后端处理用户身份、部门权限和业务请求，Pi 服务负责装配模型、Agent 和工具。多个专职 Agent 也沿用同样的装配方式，由业务代码安排它们怎样协作、怎样校验结果。



这就是序篇架构图里 Pi Runtime 所处的位置。它负责模型与工具之间的执行过程，制度数据、会话记忆、业务规则和任务管理则由外面的产品系统负责。本文跑通的这个小示例，就是后面逐步组装完整系统的起点。

行政 Agent 写到简历上就是这样：



如果你想找一个已经实现好的、可以直接写在简历上的、基于pi agent开发的自进化multi-agent项目，可以用我这个。

项目的详细介绍见《我做了一个自进化Agent项目，0实习的同学也有救了》。

扫码加入知识星球即可获取：



本篇先把模型连接和一次问答跑通。接下来的几篇，我们继续在这个示例上加工具、组织多个 Agent，再接入业务后端，让它逐步具备处理真实请求的能力。

下一篇预告 | 02 组装 Pi: Tool、Context 与输出合约

现在这个 Agent 能回答问题，还不能查学校的制度。下一篇，我们给它加上第一个制度查询工具，看看模型怎样发起调用、工具结果怎样进入下一轮请求，以及怎样让回答带上可以核对的依据。

参考资料

- [Agent Core API 与事件说明](#)
- [Coding Agent SDK](#)
- [RPC 协议](#)

